

Exp 9

Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to analyze data and generate reports for sample datasets

1) Step 1: Access HUE Web Interface

- Open browser and go to: <http://localhost:8888>
- Login with username and password

2) Step 2: Upload a Sample Dataset

- Go to file browser in hue
- Click upload and select a CSV or text file.
- Example: students.csv
- File gets saved to HDFS, eg:
/user/hue/students.csv

```
id,name,department,marks
1,John,CSE,85
2,Alice,ECE,90
3,Bob,CSE,88
4,David,EEE,76
```

students.csv

3) Step 3: Create a table in Hive or Impala

- Go to “tables” -> create table from file.
- Choose the uploaded file (students.csv)
- Define:
 - Table name: students
 - Columns: id INT, name STRING, department STRING, marks INT
 - File format: CSV
- Finish the wizard. Table gets created in Hive/Impala metastore.

4) Step 4: Query the table to analyze data

Go to **Query Editor** in HUE → Choose Hive or Impala → Write SQL queries like:

- a) View all data: `SELECT * FROM students;`
- b) Find average marks by department:
`SELECT department, AVG(marks) AS avg_marks`
`FROM students`
`GROUP BY department;`
- c) Count students in each department
`SELECT department, COUNT(*) AS student_count`
`FROM students`
`GROUP BY department;`

Click **Execute**. The result will be displayed in tabular format.

5) Step 5: Generate Reports and Visualizations

- After running a query, click on "**Chart**" option near results.
- Choose:
 - ✓ **Bar chart** (for department vs student count)
 - ✓ **Pie chart** (for percentage distribution)
 - ✓ **Line chart** (if time series data)
- Customize titles, axis labels, colors.
- Click **Save or Export** the report (PDF, PNG, CSV).

Exp 8

Implement a word count program in Hadoop and Spark.

Hello Hadoop
Hello Big Data
MapReduce is powerful

One line of input data
at a time

- Mapper is responsible for processing the input data line by line and generate key and value pairs.

Mapper

- Reads Input
- splits each line into words using “space” as a split() method value.
- Checks for Non-Empty Words: Skips empty strings caused by multiple spaces.
- Emits Key-Value Pairs

(Hello, 1)
(Hadoop, 1)
(Hello, 1)
(Big, 1)
(Data, 1)
(MapReduce, 1)
(is, 1)
(powerful, 1)

**Shuffle and sort
phase** (which
will be
happened
internally just
after the
mapper phase
by the hadoop)

(Big, [1])
(Data, [1])
(Hadoop, [1])
(Hello, [1, 1])
(is, [1])
(MapReduce, [1])
(powerful, [1])

- The **Reducer** in Hadoop MapReduce is responsible for **aggregating and summarizing** the intermediate output generated by the **Mapper**.
- Its primary purpose is to **process grouped key-value pairs** after the shuffle and sort phase and produce the **final output**.



```
(Big, [1])
(Data, [1])
(Hadoop, [1])
>Hello, [1, 1])
(is, [1])
(MapReduce, [1])
(powerful, [1])
```

Reducer

- Receives **key-value pairs** where the key is a word, and the value is a list of counts.
- Iterates over the list of counts for each word.
- Adds all values to find the total count for each word.
- Writes the word and its total count as the final output.



```
Big      1
Data     1
Hadoop   1
Hello    2
is       1
MapReduce 1
powerful 1
```

- MapReduceWordCount
 - > JRE System Library [JavaSE-1.8]
 - src
 - com.mapreduce.wordcount
 - > WordCount.java
 - > Referenced Libraries

```
C:\Windows\system32>cd \
C:\>cd hadoop
C:\hadoop>cd sbin
C:\hadoop\sbin>start-all.cmd
```

```
C:\hadoop\sbin>jps
20464 Jps
9888 ResourceManager
13236 DataNode
13512 NameNode
13004 NodeManager
```

```
C:\hadoop\sbin>hadoop fs -mkdir /input_folder

C:\hadoop\sbin>hadoop fs -put E:\Inputwords.txt /input_folder

C:\hadoop\sbin>hadoop dfs -cat /input_folder/Inputwords.txt
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
Hello Hadoop
Hello Big Data
MapReduce is powerful
```

Inputwords - Notepad

File Edit Format View Help

Hello Hadoop
Hello Big Data
MapReduce is powerful

```
C:\hadoop\sbin>hadoop jar E:\wordcount.jar com.mapreduce.wordcount/WordCount /input_folder/Inputwords.txt /output_folder
```

```
map 0% reduce 0%
map 100% reduce 0%
map 100% reduce 100%
```

```
C:\hadoop\sbin>hadoop dfs -cat /output_folder/*
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
Big      1
Data     1
Hadoop   1
Hello    2
MapReduce      1
is         1
powerful      1
```

```
package com.mapreduce.wordcount;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
public class WordCount
{
    public static class WordMapperClass extends Mapper<Object, Text, Text, IntWritable>
    {
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            for (String w : value.toString().split(" "))
            {
                if (w.length() > 0)
                {
                    context.write(new Text(w), new IntWritable(1));
                }
            }
        }
    }
}
```

```

public static class WordReducerClass extends Reducer<Text, IntWritable, Text, IntWritable>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        int sum = 0;
        for (IntWritable val : values)
        {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception
{
    Job job = Job.getInstance(new Configuration(), "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(WordMapperClass.class);
    job.setReducerClass(WordReducerClass.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

```
Hello , I am from Bengaluru  
Hello , I am from Chikkaballapur  
I am studying in SJCIT SJCIT is in Chikkaballapur
```

sample.txt

1. Create the sample text file in one of the directory.

```
from pyspark import SparkContext  
sc = SparkContext("local","WordCount")  
text_file = sc.textFile("file:///home/thejaschandra/Documents/sample.txt")  
counts = text_file.flatMap(lambda line: line.split(" ")) \  
                    .map(lambda word: (word,1)) \  
                    .reduceByKey(lambda a,b: a+b)  
output = counts.collect()  
for (word,count) in output:  
    print(f"{word}:{count}")  
sc.stop()
```

2. Type the code in texteditor and save with .py extension.
3. Execute the code by running the command **python3 filename.py**. And also, .txt and .py files should be in the same directory.
Example: python3 wordcount.py

How to check whether the spark has been installed or not?

- 1) pyspark --version

How to install PySpark via pip

- 1) pip3 install pyspark
- 2) pyspark
- 3) To exit the shell, type exit()
- 4) pyspark --version

```
thejaschandra@Aravinda:~/Documents$ python3 wordcount.py
25/05/17 18:17:46 WARN Utils: Your hostname, Aravinda resolves to a loopback address: 127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
25/05/17 18:17:46 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/05/17 18:17:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Hello:2
,:2
I:3
am:3
from:2
Bengaluru:1
Chikkaballapur:2
studying:1
in:2
SJCIT:2
is:1
```

Output of wordcount program using spark